

Kalman Filter on an FPGA: Technical Report

Oct 4, 2026

Abstract

The app's 4-state Kalman filter now runs as FPGA hardware that matches the original to within $3\text{e-}9$ m/s on every recorded run. The filter is ReferenceKF4 from the ground-speed-sensor repo: it fuses an IMU and optical flow into forward and sideways velocity. I implemented it as a small fixed-point coprocessor in Verilog, with the filter as microcode, compiled for a Cyclone V in Quartus Prime 25.1 and simulated at clock level in ModelSim and Icarus Verilog.

- **Accuracy.** Against a double-precision Python port, the fixed-point design differs by at most $2.6\text{e-}9$ m/s across 53 recorded runs. The simulated Verilog matches its bit-exact Python model in every check I ran.
- **Fidelity to the app.** The Python port reproduces the phone's own est.csv to about $1\text{e-}7$ m/s on runs recorded with the final filter.
- **Cost.** 1,733 ALMs (9 %), 10 DSP blocks and 12 RAM blocks. The 50 MHz clock is met with +2.4 ns of setup slack.
- **Speed.** A predict takes 359 clocks ($7.2\text{ }\mu\text{s}$) and a flow update 1,245 clocks ($24.9\text{ }\mu\text{s}$) at 50 MHz.
- **Integration.** The dashboard can recompute its estimate on the simulated FPGA (`--fpga`) from the raw sensor columns of each telemetry frame.

Not done: the design has not run on a physical board, the zero-velocity detector stays on the phone, and a host (Python) still sequences the commands. Section 9 lists these.

Background and goals

The goal was to move the project's Kalman filter from software to FPGA hardware and show that it behaves identically. The project (MHacks 2026) turns an iPhone on a cart into an optical ground-speed sensor. The camera and torch watch the floor, phase-correlation optical flow gives velocity, and the IMU, GNSS and LiDAR height complete the picture. A 4-state Kalman filter fuses them on the phone and streams the result to a laptop dashboard.

The filter exists as Swift code in

GroundSpeedKit/Sources/KalmanCore/ReferenceKF4.swift. Its header documents every step precisely enough that a port can match it to $1\text{e-}6$. The task was to build that filter in hardware with Quartus, simulate the FPGA, and run the dashboard's estimate on it.

Scope of this report: the FPGA side only, meaning the Verilog, the microcode generator, the models used to verify it, the Quartus results and the dashboard hook. The Swift app, the camera pipeline and the iOS code are unchanged.

What the filter receives. The phone records raw streams: IMU at 100 Hz, flow at about 240 Hz, GNSS at about 1 Hz, plus the zero-velocity (ZUPT) events its detector fires. The repo's data/phone_runs/ holds 55 such runs together with the phone's own filter output (est.csv), which gave a ground truth for the port.

The filter being implemented

The state is four numbers: forward velocity v_x , sideways velocity v_y , and the accelerometer biases b_x and b_y , with a 4x4 covariance P that records how uncertain they are. Two kinds of step alternate.

Predict runs on every IMU sample, using acceleration (a_x, a_y) , yaw rate r and the time step dt . Both velocities are advanced from the old values:

$$v_x' = v_x + dt(a_x - b_x + r), \quad v_y' = v_y + dt(a_y - b_y - r)$$

The covariance becomes $F P F^T + Q$, where $F = I + dt \cdot \begin{bmatrix} 0 & r & -1 & 0 \\ -r & 0 & 0 & -1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$. Q adds $q_{\text{accel}} \cdot dt$ to the velocity terms and $q_{\text{bias}} \cdot dt$ to the bias terms. dt is taken from timestamps and capped at 0.1 s.

Measurement updates correct the state with one scalar measurement at a time, using the Joseph-form covariance update so that P stays positive:

$$K = P H^T / S, \quad S = H P H^T + R, \quad x \leftarrow x + K \nu, \quad P \leftarrow (I - K H) P, \quad (I - K H)^T + R, \quad K K^T$$

Update Measurement	Notes
Flow v_x, v_y from the camera	$R = r_{\text{flow_base}} \cdot (\text{psr_ref} / \max(\text{PSR}, \text{psr_min}))^2$. Skipped if $\text{PSR} < 8$. x then y updates
GNSS speed	Only if speed > 1 m/s and estimated speed > 1 m/s. H points along the current velocity
ZUPT zero velocity on both axes	$R = 1e-4$. Never gated

Outlier gate. A flow or GNSS sample is rejected when its squared innovation exceeds 9 times its predicted variance (3 sigma). The gate is disarmed after a reset until the first

accepted measurement, and 12 rejected flow samples in a row trigger a recovery that re-opens the velocity covariance, so a diverged filter can lock back on.

Architecture

The filter is a program running on a small fixed-point processor, not a hard-wired circuit. The Verilog implements the processor; the Kalman math is the 857-instruction program stored in its ROM.

[embed: node/69489622-acf4]

A command (reset, predict, flow, GNSS or zero-velocity update) is started by number. The sequencer jumps to that command's entry in the ROM and then steps through instructions by itself until a halt, at which point done pulses and the host reads the outputs.

Part	What it does
Program ROM (kf_rom.v)	A lookup table of 36-bit words: opcode plus three register addresses. Generated from kf_isa.py, never edited by hand
Register file	1,024 words of 56 bits, stored twice so both operands are read in one clock. Holds the state, the 4x4 covariance, the inputs, the constants and temporaries
Multiply-accumulate	One 56 x 56 multiplier into a 120-bit accumulator. A dot product is CLR, one MAC per term, then STA (one rounding)
Divider and square root	Restoring divider at 2 quotient bits per clock; bit-serial square root for GNSS speed
Sequencer	Read, execute and write-back states, 3 clocks per instruction (branches and CLR take 2); one link register for CALL and RET

Instruction set. ADD, SUB, MUL, MAX, AVG, MOV, DIV, SQRT, the accumulator operations CLR, MAC, MACN and STA, and the control operations JMP, BLT, BGE, CALL, RET and HALT. Instructions never contain sensor values: they name registers, and the host writes each new sample into the input registers before starting a command.

Registers the host uses. Inputs: dt, a_x, a_y, yaw rate, flow v_x and v_y, flow quality, GNSS speed and accuracy. Constants: the noise and gate settings of the filter. Outputs: v_x, v_y, b_x, b_y, the four diagonal variances (the full P matrix is readable) and a status word (0 accepted, 1 gated, 2 skipped).

Host and coprocessor split. The host handles time (dt, the 0.1 s cap, initialising on the first sample), converts floats to fixed point, decides which commands each sample needs, takes the square root for sigma and integrates distance. This mirrors the Swift app, where the sensor-fusion engine handles time, ZUPT and distance and the filter class does the filtering.

Number format and arithmetic

Every value is a signed 56-bit integer with 40 fractional bits (Q16.40): a value v is stored as $\text{round}(v \cdot 2^{40})$. The range is $\pm 32,768$ and the step is $9\text{e-}13$. All arithmetic saturates instead of wrapping, and multiplies round to nearest.

The width came from measurements, not a guess. I first tried narrower formats in the Python model and replayed the recorded runs:

Format	Result on recorded runs	Why
Q12.36 in 48 bits	up to $4\text{e-}5$ m/s error	The flow quality (PSR) reaches 3,000+ and saturated at 2,048
Q8.40 in 48 bits	up to $1.8\text{e-}2$ m/s error	PSR saturated at 128
Q16.40 in 56 bits	under $3\text{e-}9$ m/s error	Range covers PSR; 40 fraction bits resolve $q_{\text{bias}} \cdot dt = 1\text{e-}7$ added to a covariance of about $1\text{e-}2$

Fixed point also forced a second change. An early version used one reciprocal $1/S$ and multiplied it into each gain term, which saved divisions. With P and R both tiny, S falls to about $2.5\text{e-}7$, so $1/S$ reaches $4\text{e}6$ and overflows Q16.40; the filter's covariance then stopped shrinking. The final design divides each gain term separately (4 divides per update), which costs about 150 clocks and removes the problem.

Two accuracy details are deliberate. A dot product such as $P \cdot h$ accumulates in a 120-bit register with a single rounding at the end. The flow and ZUPT updates use the Joseph form written out for $H = e_x$ and e_y , computing only the upper triangle of P and mirroring it, which roughly halves the instruction count compared with the general form.

Verification

The design is checked through a chain of models, each compared with the one before it, so any error has a place to hide only in one link.

Link Compared		Result
1	Python float port (kf_ref.py) vs the phone's est.csv	1e-7 m/s on runs recorded with the final filter (from 2026-10-03_22-18-56); distance matches to 3 decimals
2	Fixed-point model (kf_isa.py) vs float port, all 53 recorded runs (up to 3,000 IMU ticks each)	worst velocity difference 2.6e-9 m/s, sigma 6e-10 m/s, distance 5e-9 m
3	Verilog in ModelSim vs the fixed-point model, bit for bit	identical outcomes, outputs and the whole 197-register file on every test run
4	Quartus post-fit netlist in ModelSim vs the model	bit-exact on sequences of 1 to 7 commands; the free ModelSim crashes on longer ones

Golden model. kf_ref.py is a line-for-line Python port of ReferenceKF4, ZuptDetector, DistanceIntegrator and the replay merge order. Runs started mid-motion are seeded from the phone's logged filter_state event.

RTL tests. check_rtl.py feeds identical command sequences to the Python processor model and to the simulated Verilog. The sequences are random stress tests with saturating inputs, divide-by-zero, GNSS gating and lockout recovery, plus slices of real runs. In the quick configuration I ran (2 stress seeds of 150 commands, 2 real slices of 40 and 60 ticks), every outcome and the full register file matched exactly. The full configuration (6 seeds, longer slices) is written but I did not run it.

Waveform comparison. make_wave.py replays 80 IMU ticks (0.8 s) of run 2026-10-03_22-22-18, 190 flow updates and no ZUPT or GNSS, as 271 commands. ModelSim's recorded waveform is compared with the Python filter after every command:

Signal	Worst difference	Typical (median)
v_x	2.6e-10 m/s	5e-12
v_y	3.3e-10 m/s	5e-12
sigma_vx	1.8e-10 m/s	2e-11
bias_x	9.4e-10 m/s ²	4e-10

The differences are rounding: the FPGA works in steps of $9e-13$, Python in 64-bit floats. Neither is compared with the cart's true speed. The check shows the two filters agree, not that either is correct; the phone's est.csv (link 1) covers the Swift original.

Figure: the 271 commands of the waveform run. The pink line is the Python filter, the teal line the values ModelSim recorded; the bottom panel is the absolute difference on a log axis, far under the $1e-6$ pass bar. The first command is the seed, so its row shows the FPGA's reset state before the seed is applied and is excluded from the error.

One limit: the recorded runs contain no GNSS samples, so the GNSS update is verified by the random stress tests only, not on real data.

Quartus implementation

The design compiles cleanly in Quartus Prime 25.1 Lite for the Cyclone V device 5CEBA4F23C7, uses under 10 % of its logic and meets timing at 50 MHz. I took the device from the earlier kf_fpga project; it is not chosen for a specific board. The compile took 10 minutes of wall time (74 minutes of CPU).

Resource	Used	Available	Share
Logic (ALMs)	1,733	18,480	9 %
Registers	720		
Block RAM	114,688 bits in 12 M10K blocks	3,153,920 bits	4 %
DSP blocks	10	66	15 %

The block RAM holds the two copies of the register file. The DSP blocks build the 56×56 multiplier. The host port (about 140 signals) is routed as virtual pins, because no board link exists yet.

Timing. Worst setup slack at the 20 ns period is +2.4 ns (slow corner, 0 °C), so the longest path is about 17.6 ns. Hold slack is at least +0.12 ns in every corner. A rough upper bound for the clock is therefore in the high 50s of MHz; I did not push the constraint to find it.

Latency. An instruction takes 3 clocks (branches, CLR and NOP take 2); a divide adds 48. Whole commands, measured in the simulation:

Command	Clocks	Time at 50 MHz
---------	--------	----------------

PREDICT	359	7.2 μ s
FLOW update	1,245	24.9 μ s
ZUPT	1,180	23.6 μ s
FLOW rejected by the gate	107	2.1 μ s
RESET	87	1.7 μ s

Outputs become final at different times inside a command. Tracing the program, v_x and v_y are final after about 30 clocks of a predict (0.6 μ s) but the covariance only after about 310 clocks. In a flow update, v_x is final after about 880 clocks and the covariance after about 1,190. A full 100 Hz step (one predict plus about 2.4 flow updates at the recorded rates) takes about 66 μ s, under 1 % of the 10 ms between IMU samples. These are clock counts from simulation; no board measurement exists.

Dashboard integration and simulation speed

With `--fpga`, the dashboard throws away the phone's estimate and recomputes it on the simulated FPGA from the raw columns of the 72-byte v2 telemetry frame. It acts as the board's host CPU. Per frame it runs one predict (a_x , a_y , gyro z , dt from timestamps), one flow update when the frame carries a new flow sample (FLOW_OK set and the flow values changed), and a zero-velocity update while the phone's ZUPT bit is set. It then overwrites v_x , v_y , sigma and distance in the frame and sets the FLOW_GATED and FILTER_INIT status bits from the FPGA's own outcomes. The phone's values are kept as `frame["phone"]`.

The implementation is `dashboard/gsdash/fpga_filter.py` plus a small hook in `server.py`. The simulator runs as a child process (Icarus Verilog) and talks over a pipe through the testbench `tb_stream.v`, which reads register-write, run and register-read requests line by line. ModelSim cannot be driven interactively through a pipe, so offline replays use ModelSim in one batch launch and live mode uses Icarus.

Speed. The simulator manages about 80,000 to 100,000 FPGA clocks per second, so a flow update costs about 13 ms of wall time and a predict about 3 ms. A live 50 Hz stream ran with a load of 76 to 85 % of one core, while a Quartus compile was running on the same machine; with the fake phone simulator I saw it keep up at 49.6 frames per second. If the stage falls behind, queued frames are only predicted so the display stays current.

Replaying 300 IMU ticks of a recorded run in ModelSim takes 12 s.

These figures describe the simulation, not the chip: the real FPGA would be busy under 1 % of the time. In a real system the link between host and FPGA would dominate, because each frame needs several register writes and reads.

Tested with the repo's fake phone simulator and an automated test suite (7 new tests, all passing); not tested with a real phone. One existing dashboard test, `test_e2e.test_flow`, fails on this Windows machine with or without these changes.

Limitations and future work

The design is verified in simulation only; the items below are what stands between it and a self-contained sensor.

Limitation	Effect	Next step
Not run on a board; no UART/SPI bridge or pin assignments	Timing and area are Quartus estimates; the host port is on virtual pins	Add a link front end and a board top level, assign the real clock pin
A Python host sequences every command	The loop (convert, write, run, read) would dominate on hardware	Move the sequencing, dt, distance integration and the square root for sigma into Verilog
ZUPT detector stays on the phone	Only the zero-velocity update runs on the FPGA	Port the stationary detector (variance of acceleration magnitude over a short window, plus flow speed)
GNSS update not exercised on real data or in the dashboard	Verified by stress tests only; the dashboard frame has no GNSS speed	Add GNSS speed to the frame; record an outdoor run
Gate-level simulation covers 1 to 7 commands	The free ModelSim crashes on longer netlist runs	Use a full Questa licence or a different simulator for netlist replay
Inputs come from the phone	The FPGA never sees analog signals; the phone's ADCs digitise the sensors	Either add a front end for the phone's link, or wire an SPI IMU, optical-flow and height sensor (different hardware and retuning)
The filter is microcode on a general processor	Slower than a hand-wired datapath, but already uses	Not needed for 100 Hz; revisit only if the rate rises sharply

under 1 % of the time
budget

No NIS value is reported (the gate compares squared innovation against gate times S , avoiding a division), so diagnostics that read NIS from the Swift filter have no FPGA equivalent.